`DbContext.ChangeTracer.DebugView`, as described in the documentation at http://mng.bz/8rlz.

With the ability to update recipes, you're almost done with your recipe app. "But wait!" I hear you cry. "We haven't handled the *D* in *CRUD: delete*!" That's true, but in reality, I've found only a few occasions to delete data. Let's consider the requirements for deleting a recipe from the application:

- You need to provide an API that deletes a recipe.
- After a recipe is deleted, it must not appear in the recipe list and can't be retrieved.

You could achieve these requirements by deleting the recipe from the database, but the problem with data is that when it's gone, it's gone! What if a user accidentally deletes a record? Also, deleting a row from a relational database typically has implications on other entities. You can't delete a row from the `Recipe` table in your application, for example, without also deleting all the `Ingredient` rows that reference it, thanks to the foreign-key constraint on `Ingredient.RecipeId`.

EF Core can easily handle these true deletion scenarios for you with the `DbContext .Remove(entity)` command, but often what you mean when you find a need to delete data is to archive it or hide it from the UI. A common approach to handling this scenario is to include some sort of "Is this entity deleted?" flag on your entity, such as the `IsDeleted` flag I included on the `Recipe` entity:

```
public bool IsDeleted { get; set; }
```

If you take this approach, deleting data suddenly becomes simpler, as it's nothing more than an update to the entity—no more problems of lost data